PrepLang: a data-aware compiler for data-preparation pipelines that reuses its proofs on new batches

Nikhil V

School of Computer Science and Engineering, Vellore Institute of Technology, Vellore 632014, Tamil Nadu, India

Abstract

A data-preparation pipeline is compiled when its input already exists, and then run again on later batches the compiler never saw. PrepLang, a compiler for a small language of table and image pipelines written in C++17, uses both facts. It reads the input at compile time (every CSV cell, Parquet footers, image headers), proves or refutes the pipeline's data contracts before any data is processed, and removes the checks and operations the data proves redundant. Every removal stays in the generated pandas and OpenCV program behind a guard that holds the facts about the input it depends on. The program runs unchanged on new batches, runs a removal again only on a batch that breaks those facts, and then gives the result a fresh compilation would give. On 238 later months of real data, three compiled pipelines handled 93% as a fresh compilation would and none wrongly. The repository holds 300 test checks with sanitizer, coverage and fuzzing jobs, Lean 4 proofs for a core of the analysis, and one script that reproduces every number in this article.

Keywords: domain-specific language, data-aware compilation, abstract interpretation, speculative optimization, deoptimization, data validation

1. Motivation and significance

Mistakes in data preparation reach the trained model without an error [1]. A preparation pipeline is usually a script over pandas [2] and OpenCV [3], and it cannot know, until it runs, whether a column it normalizes holds numbers or whether the images it saves as PNG hold float pixels. Validators such as TFDV [4], Deequ [5], Great Expectations [6] and pandera [7] check the data, but apart from the code that transforms it. In batch preparation the input

Email address: atnikhil06@gmail.com (Nikhil V)

Nr.	Code metadata description	Please fill in this column
C1	Current code version	v2.0.0
C2	Permanent link to code/repository used for this code version	`https://github.com/Nikhi00718/preLang/tree/v2.0.0`
C3	Permanent link to Reproducible Capsule	Docker image `ghcr.io/nikhi00718/preplang:2.0.0` (`Dockerfile` in the repository); `eval/reproduce.sh check` builds, tests and regenerates every number of this article from the committed results
C4	Legal Code License	MIT
C5	Code versioning system used	git
C6	Software code languages, tools, and services used	C++17 (no external libraries); Python 3 with pandas, NumPy, PyArrow, OpenCV; Lean 4; CMake
C7	Compilation requirements, operating environments and dependencies	CMake 3.16+, a C++17 compiler (tested in CI: Visual Studio 2022 on Windows, GCC on Ubuntu and macOS); Python 3.12+ with pandas (2.0+, 3.0 tested), PyArrow, OpenCV; Lean 4.34.1 (optional, for the proofs)
C8	If available, link to developer documentation/manual	`README.md` (user manual), `LANGUAGE.md` (reference), `ARCHITECTURE.md`, `CONTRIBUTING.md`; the READMEs of `eval/`, `paper/`, `proofs/`, `docs/`
C9	Support email for questions	atnikhil06@gmail.com

Table 1: Code metadata (mandatory)

already exists when the pipeline is compiled, and PrepLang reads it. F# type providers read a sample to obtain types [8]; PrepLang reads every CSV cell, Parquet footer and image header to obtain values, follows types, missing values, value ranges and image properties through the pipeline, proves or refutes each contract before any data is processed, and removes the checks and operations the data proves redundant.

These proofs hold for one batch, and the pipeline runs again next month on a batch with the same layout and other values. For every removal PrepLang therefore computes the facts about the batch that the removal depends on (its *item certificate*), and the generated program checks them on each new batch. A removal whose facts hold stays removed. A removal whose facts the batch breaks runs again, as code does after deoptimization in dynamic compilers [9, 10]; this article calls it *selective deoptimization*. A batch is refused only for facts that no run-time code can replace, such as a changed column type. A user writes a `.prep` program, compiles it against the current batch, reads the diagnostics and deploys the generated program for every later batch.

Table 2 compares PrepLang with the closest tools. Validators know the values but check them apart from the code and again on every batch; compile-time analyses follow the code but know only declared or inferred structure. We found no other tool that decides value contracts at program points from the real data and carries those decisions to later batches.

Table 2: PrepLang and the closest tools. *Values*: missing values and value ranges.

Tool	Decides at	Knows about the data	Follows code	On a new batch
TFDV [4], Deequ [5], GX [6], pandera [7]	run time	values, from each batch	no	validates it again
F# type providers [8]	compile time	types, from a sample	no	–
Zhuang and Lu [11]	compile time	column types, from the code	yes	–
Gerstl et al. [12]	compile time	R data-frame shapes, optionally from the data	yes	–
Mirji [13]	compile time	declared schemas	yes	checks the output schema
PrepLang	**compile time**	**types and values of every cell; image headers**	**yes**	**admits by certificate; reruns only the removals it breaks**

2. Software description

2.1. Software architecture

The compiler (`src/`, about 11,800 lines of C++17 without external libraries) runs lexer, LL(1) parser, semantic analysis, IR, optimizer, certifier and code generator in turn (Fig. 1). The *schema* readers scan CSV files in parallel and read Parquet footers and PNG and JPEG headers. The *semantic analyzer* computes the abstract dataset state (types, missing values, value intervals, image properties) and decides every contract. The *optimizer* removes, speculatively, what the state shows to have no effect. The *certifier* reruns the analysis under weakened input facts, without reading data, to find the admission certificate and one item certificate per removal. The *back end* generates one Python program that admits a batch and guards every removed instruction with `deopt(i)`, and writes the certificates to `preplang_contract.json`.

The repository also holds `proofs/` (1477 lines of Lean 4 [14]). They prove two things: for any sound and monotone analysis, the generated program gives the output of a fresh compilation on every batch it admits; and the analysis of a core of the language is sound and monotone. Joins, groups, dates and images are outside that core. Joins and groups are checked by the differential test of Section 4, dates and images only by unit tests and the examples. The other folders are `tests/` (300 checks in 10 suites), the pandas

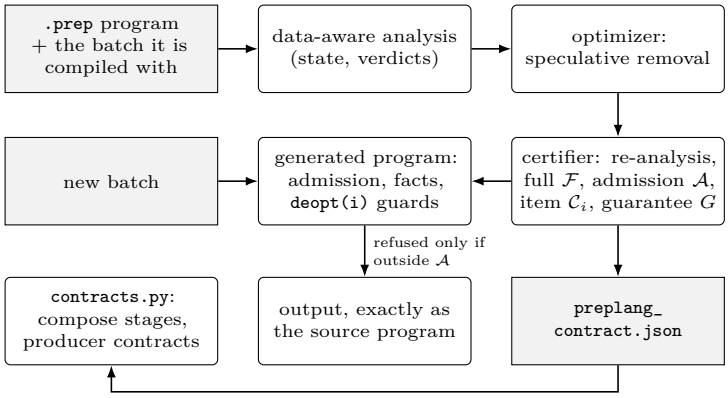


Figure 1: How PrepLang compiles once and reuses the result. The certifier re-runs the analysis with weaker input facts, reading no data; on a new batch, the generated program runs a removed instruction i only if the batch breaks its item certificate $\mathcal{C}_i$.

lifter (`frontend/`), the tools below (`tools/`), a VS Code extension and the evaluation (`eval/`).

PrepLang installs with `pip` from the repository (commands `preplang`, `preplang-lift`, `preplang-contracts`, `preplang-release-facts`) or runs from the `Dockerfile`. Continuous integration builds and tests the compiler on Windows (MSVC), Ubuntu and macOS (GCC), compiles every example, runs the tests under AddressSanitizer and UndefinedBehaviorSanitizer with a coverage gate, fuzzes the three readers of untrusted bytes (`.prep`, CSV, Parquet footers) with libFuzzer, and checks the Lean proofs. `LANGUAGE.md` documents the language, the command line and the generated program; `ARCHITECTURE.md` maps each phase to its files.

2.2. Software functionalities

- **Compile-time checking.** Misspelled or removed columns, type errors and broken contracts (`NOT_NULL`, `BETWEEN`, `UNIQUE`, `REFERENCES`, schemas, image channels and sizes, match rates of images and labels) and lookup joins on keys of other types or with repeated values are reported with the line, the offending row and a hint. Groups (`AGGREGATE`) keep their facts.
- **Data-driven optimization.** Proven checks and operations without effect are removed, unused columns are not loaded, and image pipelines are streamed. Rewrites that change the output, such as merging two `RESIZE`s, are applied only with `--approx`.
- **Tools around the compiler.** `tools/contracts.py` checks one compiled stage's guarantees against the next stage's certificates; `frontend/pandas2prep.py` lifts the cleaning part of pandas scripts into PrepLang,

up to the first statement the language cannot express; and with `--facts` a program compiles from differentially private facts that its data owner released (`tools/release_facts.py`) without opening an input file; the generated program checks those facts on the real data, so every run stays exact.

- **Inspection.** Every phase can be printed (`--tokens` to `--python`), together with an execution plan (`--explain`), the files a run would touch (`--dry-run`) and the diagnostics as JSON (`--json`).

3. Illustrative example

```
LOAD "students.csv" AS students;
EXPECT students.age NUMBER;
EXPECT students.marks BETWEEN 0 AND 100;
EXPECT students.name NOT_NULL;
FILTER students WHERE marks >= 50;
EXPECT students.marks BETWEEN 60 AND 100;
SAVE students AS "clean.csv";
```

Compiled against an eight-row `students.csv` (from `examples/tabular/`), PrepLang proves the first three contracts. It keeps the fourth as a run-time check, since after the filter it knows only that the marks lie in $[50, 95]$. It removes the range and not-null checks speculatively, and `--certificate` shows the facts the range check needs: marks at least 0 and at most 100. Run on a later batch in which a mark is 104, the generated program re-enables the check and stops where a fresh compilation would:

```
Deoptimized line 3: EXPECT students.marks BETWEEN 0 AND 100:
  marks goes up to 104, above 100
ContractError: line 3: students.marks has values outside 0 to 100
```

A batch within the facts runs with both checks removed. A batch whose `age` holds text is refused, because generated code cannot replace an analysis of the column under another type.

The same analysis covers images, from their headers alone. In `examples/images/linked.prep`, images and their labels are linked and checked, resized, normalized to $[0, 1]$ and saved as NumPy arrays. Changing the format to PNG gives, before any pixel is decoded:

```
error[type]: cannot save FLOAT32 pixels as PNG:
             NORMALIZE at line 10 scaled them to [0, 1]
   --> examples/images/linked.prep:13:1
13 | SAVE_IMAGES imgs TO "processed/ml_ready/";
   = hint: image files store 8-bit pixels, so these would come out
```

```
almost black; add CONVERT imgs TO NPY; to keep the float
values
```

The equivalent hand-written OpenCV script runs without an error and writes almost black images.

4. Impact

PrepLang is a research prototype by one author, begun in a compiler-design course, and has no users outside the author's experiments yet. Its evidence is the evaluation in `eval/`; `eval/evaluate.py` writes every number below from the raw results.

- **Correctness.** 2,000 random programs (with joins and groups), compiled with and without the optimizer, agree cell by cell with a reference interpreter written in Python that shares no code with the compiler. Reused on a second random batch, none of 1,535 runs differs from it. This test found two bugs (column pruning, a pandas 3 incompatibility) that had made 109 optimized runs wrong, and three more seeds (6,000 programs) found a third (a statistic fill removed as a dead store, 1 wrong run); after the fixes all 6,000 agree and none of 4,646 reuses is wrong. The CSV fuzzer found an out-of-bounds read on a NUL byte. Each fix has a regression test.
- **Performance.** On a table of 200,000 rows the optimized program runs in 0.47 s instead of 0.79 s after Python start-up. An image pipeline runs as fast as a hand-written streaming script, in 10 MiB instead of the 922 MiB of a script that loads every image.
- **Optimization.** On 436 programs (the examples and programs lifted from GitHub scripts) the optimizer removes 14% of the instructions (2,080 to 1,797), 28 to 7 run-time checks and 2% of the columns read, and changes 152 programs; all 436 write the same files with and without it. The lifted scripts contain no contracts, so all removed checks come from the examples and the three monthly pipelines.
- **Real programs.** Of 44 data-preparation failures reported in GitHub issues, 18 are reported before any data is processed (6 of them only after the requirement is written as an `EXPECT`). Of 733 Titanic and House Prices scripts from GitHub the lifter translates 292; their programs reproduce a fresh compilation in 3198 of 3212 runs on new batches, refuse the other 14, and are never wrong. Of 945 traffic and bike scripts it translates 72, which reproduce a fresh run on all 3255 later real months. Most untranslated scripts fail before the language matters: 547

of the 945 never call `pd.read_csv` on a file path, which the lifter needs to find the input.

- **Comparison.** Three monthly pipelines, compiled once and run on 238 later real months (UCI traffic and bikes, NYC taxi), handle 221 (93%) as a fresh compilation would and none wrongly; the 17 they refuse are taxi months in which `passenger_count` changed type. The validators answer the same question, whether the pipeline may run on a month: each learns from the month the pipeline was compiled on, as its documentation suggests (Great Expectations profiles the columns, pandera and TFDV infer a schema), and every answer is scored against a fresh compilation of the month. Great Expectations and pandera refuse 92–93% of the months, and TFDV admits 27 months on which the program then differs. The validators were not built for this decision; the numbers show what it costs to use them for it. All 24 mistakes injected into 5 example programs are reported at compile time, where a script-style pandas program crashes on 20 and writes wrong output on 4.
- **Private compilation.** At $\varepsilon = 1$ the private facts keep every removal of the taxi and traffic pipelines but only 14% of the bikes pipeline's, where AIM [15] with its own workload keeps 54%. On the 184 Titanic scripts, 74 compilations remove instructions that their own input then re-enables, because a released fact was false. In all 64,179 runs after 4,879 private compilations no output differs from a fresh compilation.
- **Overhead.** Compiling takes a median of 28 ms and at most 8 MiB (65 MiB for a million CSV rows). Checking the certificate of a new batch changes run time by -3% to $+1\%$, within measurement noise.

`eval/reproduce.sh check` (about ten minutes, no network) rebuilds the compiler, runs every test, regenerates every number above from the committed results, and reruns small versions of the differential test and the optimization census, which the Docker image can also run. `full` (several hours) downloads the inputs and reruns every experiment, and `baselines` reruns the validators, in pinned environments. The GitHub script corpora are frozen as evaluated, each file by its URL at a fixed commit and its hash.

5. Limitations and conclusions

Joins are lookups on a key with one name in both tables (no many-to-many joins), groups have five aggregates, and only Python is generated. A changed column type is refused rather than handled. The mechanized proofs do not cover joins, groups, dates or images. The lifter translated a minority of the

GitHub scripts above. The comparison with validators uses three pipelines written by the author, and PrepLang has no outside users yet and no user study; `docs/USER_STUDY.md` is its protocol. Planned work is general joins, generated code that adapts to type changes, and that study, with people who write pandas cleaning scripts.

Declaration of competing interest

The author declares no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Funding

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

Use of generative AI

Generative AI tools assisted with implementation, literature search, evaluation scripts, analysis and drafting of this manuscript. The author reviewed all text, code and results and takes responsibility for them.

References

[1] D. Sculley, G. Holt, D. Golovin, E. Davydov, T. Phillips, D. Ebner, V. Chaudhary, M. Young, J.-F. Crespo, and D. Dennison, "Hidden technical debt in machine learning systems," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2015, pp. 2503–2511.

[2] W. McKinney, "Data structures for statistical computing in Python," in *Proc. 9th Python in Science Conf. (SciPy)*, 2010, pp. 56–61.

[3] G. Bradski, "The OpenCV library," *Dr. Dobb's Journal of Software Tools*, 2000.

[4] E. Breck, N. Polyzotis, S. Roy, S. E. Whang, and M. Zinkevich, "Data validation for machine learning," in *Proc. Conf. on Machine Learning and Systems (MLSys)*, 2019.

[5] S. Schelter, D. Lange, P. Schmidt, M. Celikel, F. Biessmann, and A. Grafberger, "Automating large-scale data quality verification," *Proc. VLDB Endowment*, vol. 11, no. 12, pp. 1781–1794, 2018.

[6] Great Expectations, "Great Expectations: data quality testing," `https://greatexpectations.io`.

[7] N. Bantilan, "pandera: Statistical data validation of pandas dataframes," in *Proc. 19th Python in Science Conf. (SciPy)*, 2020, pp. 116–124.

[8] T. Petricek, G. Guerra, and D. Syme, "Types from data: Making structured data first-class citizens in F#," in *Proc. 37th ACM SIGPLAN Conf. on Programming Language Design and Implementation (PLDI)*, 2016, pp. 477–490.

[9] U. Hölzle, C. Chambers, and D. Ungar, "Debugging optimized code with dynamic deoptimization," in *Proc. ACM SIGPLAN Conf. on Programming Language Design and Implementation (PLDI)*, 1992, pp. 32–43.

[10] A. Barrière, S. Blazy, O. Flückiger, D. Pichardie, and J. Vitek, "Formally verified speculation and deoptimization in a JIT compiler," *Proc. ACM Program. Lang.*, vol. 5, no. POPL, pp. 1–26, 2021.

[11] Y. Y. Zhuang and M.-Y. Lu, "Enabling type checking on columns in data frame libraries by abstract interpretation," *IEEE Access*, vol. 10, 2022.

[12] O. Gerstl, F. Sihler, and M. Tichy, "Inferring the shape of data frames in R programs using abstract interpretation," arXiv:2607.03889, 2026.

[13] V. Mirji, "Shift schema drift left: Policy-aware compile-time contracts for typed JVM and Spark pipelines," arXiv:2604.16986, 2026.

[14] L. de Moura and S. Ullrich, "The Lean 4 theorem prover and programming language," in *Proc. 28th Int. Conf. on Automated Deduction (CADE)*, LNCS 12699, 2021, pp. 625–635.

[15] R. McKenna, B. Mullins, D. Sheldon, and G. Miklau, "AIM: An adaptive and iterative mechanism for differentially private synthetic data," *Proc. VLDB Endowment*, vol. 15, no. 11, pp. 2599–2612, 2022.